

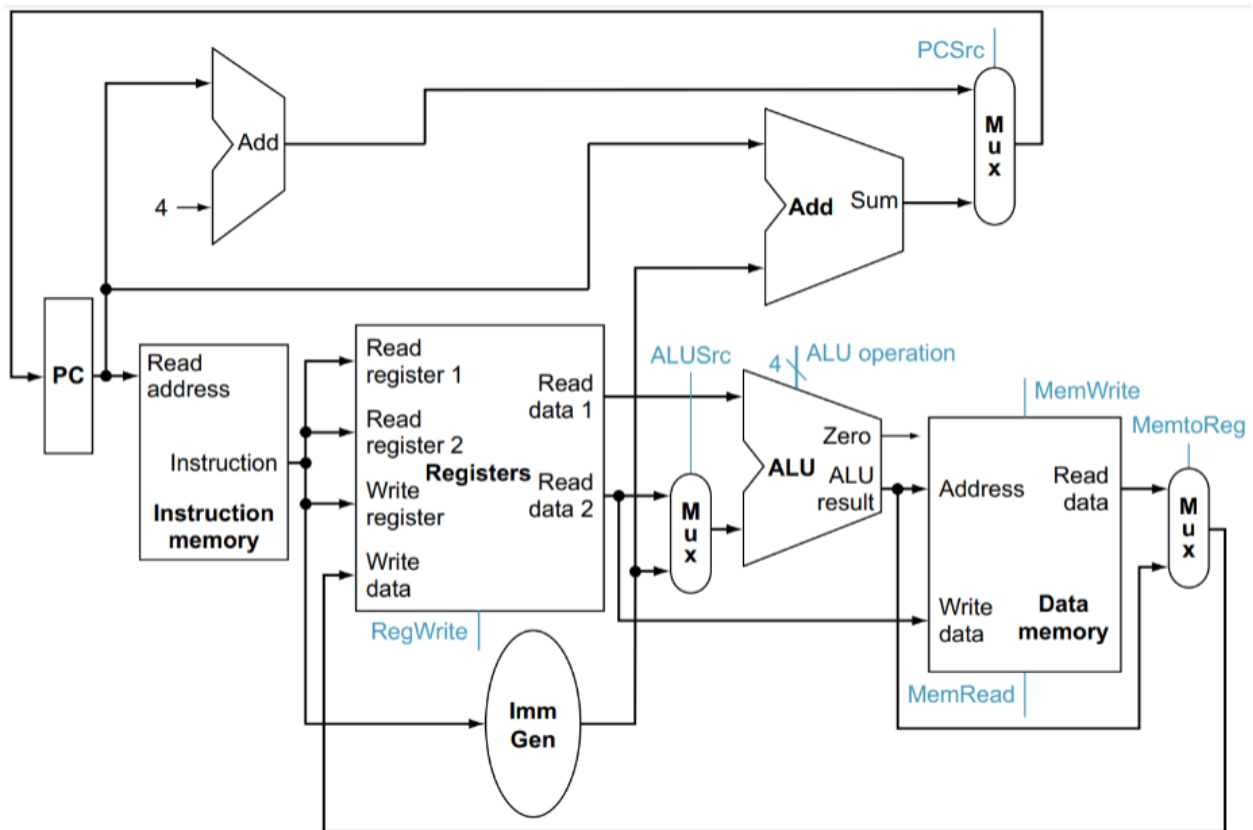
beq

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
[12] [10:5] | rs2 | rs1 | funct3 | [4:1] [11] | opcode
(7 bits immed.) (5 bits immed.)

need to calculate **branch target address** and
conditionally **choose new PC value**

- enhance “**immediate generation**” unit, add **new adder**,
add **new multiplexor**

(from text Fig. 4.11)



assuming similarly as before: memory units (read or write) - 2 ns, ALU - 1 ns, adders - 1 ns, register file (read or write) - 0.5 ns, how long would the **clock cycle time** need to be?

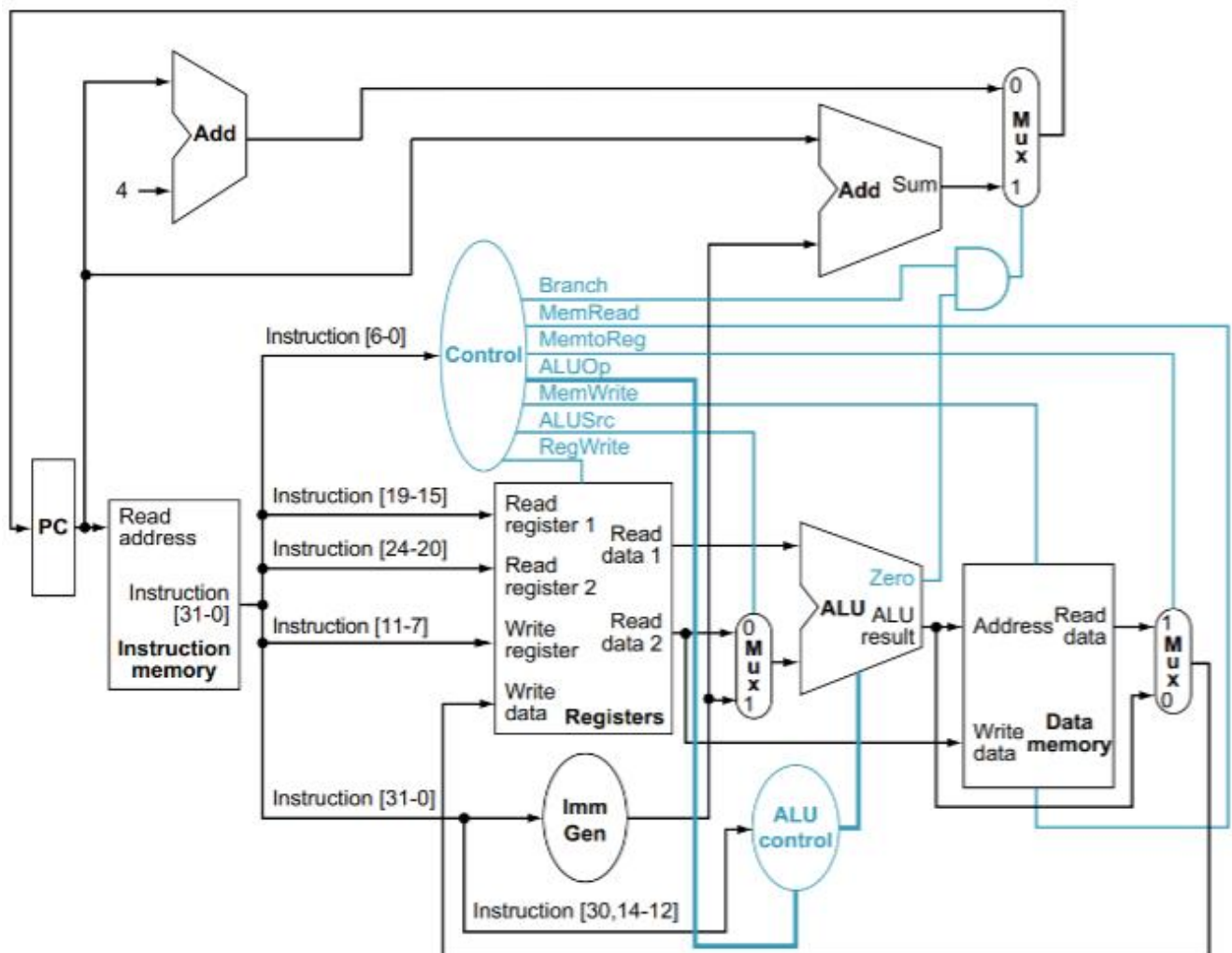
control for our single clock cycle datapath

task is to determine all the control values

- read/write enable control inputs
- selector inputs for multiplexors
- ALU control inputs

uses opcode and function fields from instruction
split into “main control” and “ALU control”

(from text Fig. 4.21)



main control

takes as input the **opcode** field, and outputs **all control values except those for the ALU**

can specify using a truth table: in the tables below, simply give the instruction names rather than the corresponding opcode field values

- use “**don’t cares**” (marked as “**X**”) when we don’t care what the value is

(based on text Fig. 4.22)

Instruction	ALUSrc	MemtoReg	RegWrite	MemRead
R-type	?	?	1	0
lw	?	?	1	1
sw	?	X	0	0
beq	?	X	0	0

Instruction	MemWrite	Branch	ALUOp1	ALUOp0
R-type	0	0	1	0
lw	0	0	0	0
sw	1	0	0	0
beq	0	1	0	1

Some values are missing in the first table ... what should they be?

ALU control

inputs: function fields, and a two bit value from “main control” that specifies whether instruction is:

- **R-type** (in which case, “ALU control” should use the function fields to determine the ALU control values) – “10” used for this case
- **“lw”/“sw”** (ALU control values should be set so that the ALU result is the addition of its two operands) – “00” used for this case
- **“beq”** (ALU control values should be set so that the ALU result is the subtraction of its two operands) – “01” used for this case

outputs: control inputs for the ALU
again, can specify using a truth table
(based on text Fig. 4.13)

	ALUOp	funct7	funct3	ALU control
(“lw”/“sw”)	00	XXXXXXXX	XXX	0010
(“beq”)	X1	XXXXXXXX	XXX	0110
(“add”)	1X	0000000	000	0010
(“sub”)	1X	0100000	000	0110
(“and”)	1X	0000000	111	0000
(“or”)	1X	0000000	110	0001